"Desktop Voice Assistant--Jarvis"

A Project Report submitted

In partial fulfillment of the requirements for the award of the degree

of

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE & ENGINEERING (DATA SCIENCE)

Submitted by

G.SAI RAHUL SIDDARTH

Regd No. 2025231496

Under the esteemed guidance of

Dr. Devarapalli Ravi Kishore, Assistant Professor, CSE

DEPARTMENT OF COMPUTERSCIENCE AND ENGINEERING (DATA SCIENCE)
GITAMSCHOOL OF TECHNOLOGY
GITAM(Deemed to beUniversity)
VISAKHAPATNAM
2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)
GITAMSCHOOLOF TECHNOLOGY
GITAM (Deemedto be University)

DECLARATION

I hereby declare that the project report entitled ***"Desktop Voice Assistant -- Jarvis"*** is an original work done in the Department of Computer Science and Engineering (Data Science), GITAM School of Technology, GITAM (Deemed to be University) submitted in partial fulfillment of the requirements for the award of the degree of B.Tech. in Computer Science and Engineering. The work has not been submitted to any other college or University for the award of any degree or diploma.

Date: 06/04/2026

Registration No	Name	Student Signature
2025231496	G.SAI RAHUL SIDDARTH	

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(DATA SCIENCE)

GITAM SCHOOL OF TECHNOLOGY

GITAM(DeemedtobeUniversity)

CERTIFICATE

This is to certify that the project report entitled "***Desktop Voice Assistant -- Jarvis***" is a bonafide record of work carried out by G.SAI RAHUL SIDDARTH, Regd No:2025231496 submitted in partial fulfillment of requirement for the award of degree of Bachelors of Technology in Computer Science and Engineering (Data Science) .

Date : 06/04/2026

Project Guide

Dr. Devarapalli Ravi Kishore

Associate Professor

TABLE OF CONTENTS

S.No.	Description	PAGENO.
1.	Abstract	1
2.	Introduction	2
3.	Aim oftheProject	3
4.	Problem Definition & Objectives	4
5.	Co ding	5-8
6.	Test Cases	9
7.	Output Screens	10
8.	Future Work	11
9.	Conclusion	12

ABSTRACT

This project presents Jarvis, a Voice Controlled Desktop Assistant built using Python. The assistant uses speech recognition to capture user voice commands and responds using Windows SAPI text-to-speech engine. Jarvis is capable of performing a wide range of tasks including telling the current time and date, fetching live weather updates, telling jokes, searching the web, opening and closing desktop applications and websites, playing music, checking battery status, taking screenshots, and reading the latest news headlines.

The system leverages several Python libraries including SpeechRecognition for voice input, win32com for speech output, requests for API communication, psutil for system information, and pyautogui for screen capture. External APIs used include OpenWeatherMap for weather data, NewsAPI for news headlines, and JokeAPI for random jokes.

The project demonstrates the practical application of voice-controlled interfaces and Python programming in building a functional desktop productivity tool.

INTRODUCTION

Voice assistants have become an integral part of modern computing, enabling hands-free interaction with devices and systems. Popular examples include Amazon Alexa, Google Assistant, Apple Siri, and Microsoft Cortana. These systems use natural language processing and speech recognition to understand and respond to user commands.

This project, Jarvis, is a desktop-based voice assistant built entirely using Python. It is designed to run on Windows operating systems and provides a range of useful features that enhance productivity and user experience. The assistant listens continuously through the microphone, processes voice commands using Google Speech Recognition, and responds verbally using the Windows SAPI voice engine.

The motivation behind this project is to demonstrate how Python's rich ecosystem of libraries can be combined to build a fully functional voice assistant without relying on paid platforms or cloud-based AI services. The project also serves as a learning exercise in integrating multiple APIs, handling real-time audio input, and building modular Python applications.

AIM OF THE PROJECT

The aim of this project is to design and develop a voice-controlled desktop assistant using Python that can:

- Accept voice commands from the user through a microphone.
- Process and recognize speech using Google Speech Recognition API.
- Respond verbally using Windows SAPI text-to-speech engine.
- Perform a variety of tasks such as fetching weather, telling time and date, searching the web, opening applications, and reading news.

PROBLEM DEFINATION AND OBJECTIVES

Problem Definition

In today's fast-paced world, users need quick and efficient ways to interact with their computers. Traditional keyboard and mouse interactions can be time-consuming for simple tasks like checking the weather, searching the web, or opening applications. There is a need for a lightweight, offline-capable voice assistant that can run on any Windows PC without requiring subscription fees or cloud-based AI platforms.

Objectives

- To build a voice-controlled desktop assistant using Python.
- To integrate speech recognition for voice input.
- To implement text-to-speech for voice output.
- To connect with external APIs for weather, news, and jokes.
- To enable opening and closing of desktop applications via voice.
- To implement useful features like screenshot, battery check, and music playback.

CODING

The project is implemented in a single Python file named jarvis.py. The code is organized into logical sections: configuration, text-to-speech setup, speech recognition, command functions, and the main loop.

Libraries Used

Library	***Purpose***
speech_recognition	Voice input from microphone
win32com.client	Windows SAPI text-to-speech API calls
requests	for weather, news, jokes
psutil	Battery status information
pyautogui	Screenshot capture
datetime, os, subprocess	System operations

Source Code

```
# jarvis.py

import speech_recognition as sr
import datetime import
win32com.client import logging
import webbrowser import os
import subprocess import
requests import random import
psutil import pyautogui
pyautogui.FAILSAFE = False
```

```
# Configuration CITY = "Visakhapatnam"
WEATHER_API_KEY = "your_openweathermap_key_here"
ASSISTANT_NAME = "Jarvis" USER_NAME = "Bhevanesh"
MUSIC_FOLDER = r"C:\Users\bhava\Music"
NEWS_API_KEY = "your_newsapi_key_here"



# Logging
logging.basicConfig(filename="log.txt",
level=logging.INFO,
                    format="%(asctime)s -
%(message)s")

# TTS Setup
speaker = win32com.client.Dispatch("SAPI.SpVoice")

def speak(text):
    print(f"{ASSISTANT_NAME}: {text}")
    logging.info(f"Jarvis: {text}")
    speaker.Speak(text)

def listen():
    r = sr.Recognizer()
    with sr.Microphone() as source:
        print("Listening...")
        r.adjust_for_ambient_noise(source,
duration=1)
        r.pause_threshold = 1
        try:
            audio = r.listen(source, timeout=10,
phrase_time_limit=10)
        except sr.WaitTimeoutError:
            return ""
    try:
```

```
        query = r.recognize_google(audio,
language="en-US")
        print(f"You: {query}")
        logging.info(f"You: {query}")
        return query.lower()
    except:
        speak("Sorry, I didn't catch that.")
        return ""

def get_time():
    return f"The current time is
{datetime.datetime.now().strftime('%I:%M %p')}"

def get_date():
    return f"Today is
{datetime.datetime.now().strftime('%A, %d %B %Y')}"

def tell_joke():
    try:
        url =
"https://v2.jokeapi.dev/joke/Any?type=single&blacklis
tFlags=nsfw,racist,sexist"
        return requests.get(url).json()["joke"]
    except:
        return "Sorry, I couldn't fetch a joke."

def get_weather():
    try:
        url =
f"http://api.openweathermap.org/data/2.5/weather?q={C
ITY}&appid={WEATHER_API_KEY}&units=metric"
        data = requests.get(url).json()
        return f"Weather in {CITY}:
{data['weather'][0]['description']},
{data['main']['temp']}C"
```

```
    except:
        return "Couldn't fetch weather."

def process_command(command):
    speak("One second!") speak("Here we go!")
    if "time" in command: speak(get_time())
    elif "date" in command: speak(get_date())
    elif "joke" in command: speak(tell_joke())

    elif "weather" in command: speak(get_weather())
    elif "bye" in command or "exit" in command:
        speak("Goodbye!")
        exit()
    else: speak("Sorry, I didn't understand.")

if __name__ == "__main__":
    hour = datetime.datetime.now().hour
    greeting = "Good morning" if hour < 12 else "Good
afternoon" if hour < 18 else "Good evening"
    speak(f"{greeting}, {USER_NAME}! I am
 {ASSISTANT_NAME}.")
    while True:
        command = listen()
        if command:
            process_command(command)
```

TEST CASES

Test No.	Voice Command	Expected Output	Result
1	What is the time	Speaks current time	Pass
2	What is the date	Speaks current date	Pass
3	Tell me a joke	Speaks a random joke	Pass
4	What is the weather	Speaks weather in Visakhapatnam	Pass
5	Search Python tutorial	Opens Google search	Pass
6	Open Notepad	Opens Notepad application	Pass
7	Open YouTube	Opens YouTube in browser	Pass
8	Close Chrome	Closes Google Chrome	Pass
9	Take a screenshot	Saves screenshot to Desktop	Pass
10	Bye	Says goodbye and exits	Pass

OUTPUT SCREENS

The following shows the terminal output of Jarvis during execution:

Time Command:

```
Jarvis: Good morning, Bhevanesh! I am Jarvis, your voice assistant. Listening... Recognizing... You: what is the time Jarvis: One second! Jarvis: Here we go! Jarvis: The current time is 10:30 AM
```

Weather Command:

```
Listening... You: what is the weather Jarvis: One second! Jarvis: Here we go! Jarvis: The weather in Visakhapatnam is clear sky with a temperature of 32.5°C
```

Joke Command:

```
Listening... You: tell me a joke Jarvis: One second! Jarvis: Here we go! Jarvis: Why do programmers prefer dark mode? Because light attracts bugs.
```

FUTURE WORK

The current implementation of Jarvis provides a solid foundation for a voice-controlled desktop assistant. The following enhancements are planned for future development:

- Face Recognition: Implement OpenCV-based face recognition to identify the user and provide personalized greetings and access control.
- Multi-language Support: Add support for Telugu and Hindi voice commands using Google Speech Recognition's multilingual capabilities.
- GUI Dashboard: Develop a graphical user interface using Tkinter or PyQt that displays Jarvis status, last command, weather, and time in real-time.
- Email and WhatsApp Integration: Enable Jarvis to send emails and WhatsApp messages through voice commands.
- Reminder and Alarm System: Implement a scheduling system to set reminders and alarms using voice commands.
- Smart Home Integration: Connect with IoT devices to control smart home appliances through voice commands.
- Offline Mode: Develop an offline speech recognition capability using Vosk or Whisper to reduce internet dependency.

CONCLUSION

This project successfully demonstrates the development of a fully functional voice-controlled desktop assistant using Python. Jarvis is capable of understanding and responding to a wide range of voice commands, performing tasks such as fetching weather information, telling jokes, searching the web, opening and closing applications, playing music, checking battery status, taking screenshots, and reading news headlines.

The project highlights the power and versatility of Python's library ecosystem in building practical applications. By integrating multiple external APIs and system-level libraries, Jarvis provides a seamless hands-free computing experience without relying on paid platforms.

The modular design of the code ensures that new features can be easily added in the future. The project serves as a strong foundation for more advanced voice assistant systems that could incorporate artificial intelligence, natural language processing, and smart home integration.

Overall, this project has been a valuable learning experience in Python programming, API integration, speech processing, and software design. It demonstrates that sophisticated desktop applications can be built using open-source tools and free APIs.